Jayawant Shikshan Prasarak Mandal's
JSPM Narhe Technical Campus
S. No. 12/2/2 and 14/9, Narhe, Tal : Haveli, Dist.: Pune – 411041
Phone : +91 20 2460 8700, 701, 702 Email : director@jspmntc.edu.in Web : www.jspmntc.edu.in
Affiliated to Savitribai Phule Pune University. Approved by AICTE New Delhi and DTE Maharashtra.
DTE Code : 6755 PUN Code : CEGP019070 AISHE Code : C-45874

Department of Computer Engineering

Savitribai Phule Pune University (SPPU)

Third Year Computer Engineering (2024 Pattern)

A Laboratory Manual for

Artificial Intelligence Laboratory (PCC304COM)

Semester I

Lab Manual

Artificial Intelligence Laboratory

Prepared By: Prof. Snehal S. Ralebhat

Curriculum Structure - Semester V

Third Year Engineering (2024 Pattern) – Computer Engineering

Course Code	Course Name	Course Type	Teaching Scheme			Examination Scheme						Credits			
			Theory	Tut	Practical	CCE	ESE	Term Work	Practical	Oral	Total	Theory	Tut	Practical	Total
PCC301COM	Artificial Intelligence	PCC	3	-	-	30	70	-	-	-	100	3	-	-	3
PCC302COM	Computer Networks	PCC	3	-	-	30	70	-	-	-	100	3	-	-	3
PCC303COM	Theory of Computation	PCC	3	-	-	30	70	-	-	-	100	3	-	-	3
PCC304COM	Artificial Intelligence Lab	PCC		-	2	-	-	25	25	-	50	-	-	1	1
PCC305COM	Computer Networks Lab	PCC		-	4	-	-	25		25	50	-	-	2	2
PEC321COM	Elective I	PEC	3	-	-	30	70	-	-	-	100	3	-	-	3
PEC322COM	Elective Lab	PEC		-	2	-	-	25	-	-	25	-	-	1	1
MDM331COM	Robotics and Automation	MDM		1	4	-	-	25	25	-	50	-	1	2	3
OLE341COM	Open Elective	OE	2	-	-	30	70		-	-	100	2	-	-	2
ELC342COM	Technical Seminar	ELC		-	2	-	-	-	25	-	25	-	-	1	1
Total			**14**	1	**14**	**150**	**350**	**100**	**75**	**25**	**700**	**14**	**1**	**7**	**22**

Elective I	
PEC321ACOM	Data Warehousing and Data Mining
PEC321BCOM	Cloud Computing
PEC321CCOM	Mobile Computing
PEC321DCOM	Embedded systems

Open Elective 1	
OEL321A	Sustainability Development
OEL321B	The Constitution of India
OEL3221C	IPR and Cyber Laws
OEL3221D	Digital Personal Data Protection

Savitribai Phule Pune University **Third Year - Computer Engineering (2024 Pattern)**		
PCC304COM- Artificial Intelligence Laboratory		
Teaching Scheme	**Credits**	**Examination Scheme**
Practical : 02 Hours/Week	01	Term Work: 25 Marks Practical : 25 Marks

Course Objectives:

1. To provide a comprehensive understanding of the fundamental concepts, principles, and methodologies of Artificial Intelligence.
2. To equip students with the ability to design and implement core AI algorithms for problem-solving, search, reasoning, and learning.
3. To develop analytical and practical skills for modeling and solving real-world problems using appropriate AI techniques.

Course Outcomes: Upon successful completion of this course, students will be able to:

- **CO1:** Explain and differentiate fundamental concepts, components, and techniques of Artificial Intelligence, including intelligent agents and AI methodologies.
- **CO2**: Design and implement AI algorithms for state-space search, heuristic search, reasoning, and basic learning tasks using programming tools.
- **CO3**: Model real-world problems as AI problems and apply appropriate AI techniques to obtain optimal or near-optimal solutions.
- **CO4**: To apply AI planning techniques for solving classical planning problems and relate them to real-world industrial AI applications.

Guidelines for Instructor's Manual

The instructor's manual is to be developed as a reference and hands-on resource. It should include prologue (about University/program/ institute/ department/foreword/ preface), curriculum of the course, conduction and Assessment guidelines, topics under consideration, concept, objectives, outcomes, set of typical applications/assignments/ guidelines, and references.

Guidelines for Student's Laboratory Journal

The laboratory assignments are to be submitted by student in the form of journal. Journal consists of Certificate, table of contents, and handwritten write-up of each assignment (Title, Date of Completion, Objectives, Problem Statement, Software and Hardware requirements, Assessment grade/marks and assessor's sign, Theory- Concept in brief, algorithm, flowchart, test cases, Test Data Set(if applicable), mathematical model (if applicable), conclusion/analysis). Program codes with sample output of all performed assignments are to be submitted as softcopy. As a conscious effort and little contribution towards Green IT and environment awareness, attaching printed papers as part of write-ups and program listing to journal must be avoided. Use of DVD containing students programs maintained by Laboratory In-charge is highly encouraged. For reference one or two journals may be maintained with program prints in the Laboratory.

Guidelines for Laboratory /Term Work Assessment

Continuous assessment of laboratory work should be based on overall performance of Laboratory assignments by a student. Each Laboratory assignment assessment will assign grade/marks based on parameters, such as timely completion, performance, innovation, efficient codes, and punctuality.

Guidelines for Practical Examination

Problem statements must be decided jointly by the internal examiner and external examiner. During practical assessment, maximum weightage should be given to satisfactory implementation of the problem statement. Relevant questions may be asked at the time of evaluation to test the student's understanding of the fundamentals, effective and efficient implementation. This will encourage, transparent evaluation and fair approach, and hence will not create any uncertainty or doubt in the minds of the students. So, adhering to these principles will consummate our team efforts to the promising start of student's academics.

Guidelines for Laboratory Conduction

List of Assignments (Part A) - Any FOUR

1. Design and implement a simple reflex agent for the Vacuum Cleaner world and analyze its rational behavior in a given environment.
2. Implement a model-based intelligent agent capable of navigating a grid environment with obstacles using internal state representation.
3. Implement A* Search for 8 puzzle problem
4. Implement Adversarial Search with Alpha-Beta pruning
5. Implement Tower of Hanoi by State Space Search
6. Implement a Sudoku solver using backtracking and constraint propagation techniques.
7. Implement Local Beam Search for The Traveling Salesman Problem (TSP).

List of Assignments (Part B) - Any FOUR

1. Use BFS to solve a planning problem. Examples can be 8 puzzle problem, Robot path planning, Blocks world etc.
2. Use DFS for problem solving. Examples can be water jug problem , Missionaries and Cannibals ,maze solving, blocks world etc.
3. Implement Backtracking search to solve the 8-Queens constraint satisfaction problem.
4. Develop a simple chatbot using predefined rules and pattern matching.
5. Design an intelligent agent that guesses a number using feedback (higher/lower).
6. Implement Goal Stack Planning for solving the Blocks World problem and achieve a specified goal configuration.
7. Develop a game-playing agent using the Minimax algorithm for optimal decision-making.

Part C - Mini-Project (In Team of 3-4 Students) - Any ONE but not limited to

1. Medical Diagnosis System (Rule-Based Expert System) using forward and backward chaining
2. Develop a simple rule-based recommendation system (e.g., movie or book suggestion system).

Index

Sr. No.	Title of Assignment
Part A (Any Four)	
1	**Simple Reflex Agent for the Vacuum Cleaner World**
2	**Model-Based Intelligent Agent for Grid Navigation with Obstacles**
3	A* Search for the 8-Puzzle Problem
4	Adversarial Search with Alpha-Beta Pruning
5	**Tower of Hanoi using State Space Search**
6	**Sudoku Solver using Backtracking and Constraint Propagation**
7	Local Beam Search for the Traveling Salesman Problem (TSP)
Part B (Any Four)	
1	**BFS for Solving a Planning Problem (8-Puzzle)**
2	**DFS for Problem Solving (Water Jug Problem)**
3	**8-Queens Problem using Backtracking Search (CSP)**
4	**Simple Rule-Based Chatbot using Pattern Matching**
5	Intelligent Agent for a Number Guessing Game using Feedback
6	Goal Stack Planning for the Blocks World Problem
7	Game-Playing Agent using the Minimax Algorithm (Tic-Tac-Toe)
Part C - Mini Project (Any One)	
1	Medical Diagnosis System (Rule-Based Expert System) using Forward and Backward Chaining
2	Rule-Based Movie/Book Recommendation System

Assignment A1

Title

Design and implement a simple reflex agent for the Vacuum Cleaner world and analyse its rational behaviour in a given environment.

1. Aim

To design and implement a simple reflex agent for the Vacuum Cleaner World and analyse its rational behaviour in a given environment.

2. Prerequisite

Basic concepts of Artificial Intelligence, notion of an agent, environment, sensors and actuators; elementary Python/Java/C++ programming.

3. Objective

- To understand the concept of an intelligent agent and its interaction with an environment.
- To study the structure and working of a simple reflex agent.
- To design and implement a simple reflex agent for the Vacuum Cleaner world.
- To analyse the rational behaviour of the agent for a given performance measure.

4. Outcome

Students will be able to design a simple reflex agent, implement condition-action rules for the Vacuum Cleaner world, and evaluate whether the agent behaves rationally under a given performance measure.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, any text editor or IDE (VS Code / PyCharm / Eclipse), Windows/Linux OS, 4 GB RAM minimum.

6. Theory

What is an Intelligent Agent?

An agent is anything that can perceive its environment through sensors and act upon that environment through actuators. Mathematically, an agent's behaviour can be described by an agent function that maps every possible percept sequence to an action. The agent program is the concrete implementation of this function running on the agent architecture (hardware/software).

Structure of Agents

AI agents are broadly classified as: Simple Reflex Agents, Model-Based Reflex Agents, Goal-Based Agents, Utility-Based Agents, and Learning Agents. This assignment focuses on the simplest of these -- the Simple Reflex Agent.

Simple Reflex Agent

A simple reflex agent selects actions based only on the current percept, ignoring the rest of the percept history. It works on the principle of condition-action rules (also called situation-action rules), typically written as: IF condition THEN action. Such agents are fast and easy to implement, but they only work correctly if the environment is fully observable, and they can get stuck in infinite loops in partially observable environments.

The Vacuum Cleaner World

The Vacuum Cleaner world is a classic toy problem used to illustrate agent design. It typically consists of two locations, A and B. Each location can be either Clean or Dirty. The agent (vacuum cleaner) perceives its current location and whether that location is dirty, and it can perform one of four actions: Suck, Move Left, Move Right, or NoOp (do nothing).

Condition-Action Rule Table

A simple reflex agent for this world can be completely described by the following rule table:

Percept: [Location, Status]	Action
[A, Dirty]	Suck
[A, Clean]	Move Right
[B, Dirty]	Suck
[B, Clean]	Move Left

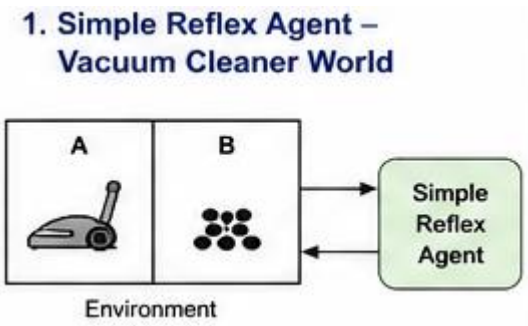
Performance Measure and Rationality

A performance measure evaluates a sequence of environment states -- for example, +1 point for every clean square at every time step, minus a penalty for every movement. An agent is rational if, for every possible percept sequence, it selects an action expected to maximize its performance measure given the evidence provided by the percept sequence and whatever built-in knowledge it has. The simple reflex agent above is rational for a fully observable, deterministic two-square world, but may behave irrationally (e.g., oscillate forever) if the world has more than two locations or is only partially observable.

Algorithm

1. Initialize the environment with random dirt status at each location.
2. Repeat for a fixed number of steps or until both locations are clean:
   a. Sense current location and its status.
   b. Apply the condition-action rule table to select an action.
   c. Execute the action and update the environment.
   d. Update the performance measure.
3. Report the final environment state and total performance score.

Diagram



Percept (Current State)	Action
Location A, Dirty	Suck
Location A, Clean	Right
Location B, Dirty	Suck
Location B, Clean	Left

7. Conclusion

The simple reflex agent for the Vacuum Cleaner world was designed and implemented successfully using condition-action rules. The agent's behaviour was traced for different initial configurations and found to be rational for the given fully-observable two-square environment, cleaning both squares in a bounded number of steps.

8. Viva Voce Questions

1. What is a rational agent? Give the general formula for rationality.
2. Differentiate between a simple reflex agent and a model-based reflex agent.
3. Why can a simple reflex agent fail in a partially observable environment?
4. What are the components of a task environment? (Use the PEAS description.)
5. Give the PEAS description for the Vacuum Cleaner world.
6. What is the difference between an agent function and an agent program?
7. Can a simple reflex agent have internal state? Justify.
8. What happens if the vacuum world has more than two rooms -- how would you extend the rule table?

Assignment A2

Title

Implement a model-based intelligent agent capable of navigating a grid environment with obstacles using internal state representation.

1. Aim

To design and implement a model-based intelligent agent capable of navigating a grid environment with obstacles using an internal state representation.

2. Prerequisite

Concepts of agents and environments; arrays/matrices; basic graph or grid representation; Assignment 1 (Simple Reflex Agent).

3. Objective

- To understand the limitations of simple reflex agents in partially observable environments.
- To study the concept of internal state and world model maintained by a model-based agent.
- To design and implement a model-based agent that can navigate a grid environment containing static obstacles.
- To analyse how the internal state helps the agent make better decisions than a purely reactive agent.

4. Outcome

Students will be able to build and maintain an internal state/model of a partially observable environment and use it to drive an agent's navigation decisions around obstacles toward a goal cell.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, matplotlib (optional, for visualization), Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Limitation of Simple Reflex Agents

Simple reflex agents work well only when the environment is fully observable -- that is, the current percept contains all information needed to choose the correct action. Many real-world environments (e.g., a robot in a room it cannot fully see) are only partially observable. In such cases, an agent that reacts to the current percept alone can make poor or repetitive decisions because it lacks memory of what it has already seen or done.

Model-Based Reflex Agents

A model-based reflex agent overcomes this limitation by maintaining an internal state that depends on the percept history. To update this internal state, the agent needs two kinds of knowledge encoded in the agent program: (1) information about how the world evolves independently of the agent (a transition model), and (2) information about how the agent's own actions affect the world (an action model). Together these constitute the agent's model of the world -- hence the name 'model-based agent'.

Working Cycle

At each time step the model-based agent: (1) receives a percept, (2) updates its internal state using the previous internal state, the transition model, the action just performed, and the new percept, (3) uses condition-action rules matched against the internal state (rather than the raw percept) to choose the next action.

Grid Navigation Problem

In this assignment the environment is represented as an N x N grid. Some cells are marked as obstacles (blocked) and cannot be entered. The agent starts at a given cell and must reach a goal cell by moving Up, Down, Left, or Right, one cell at a time, while avoiding obstacles. The agent maintains an internal grid-map (its model of the world) that it updates as it senses neighbouring cells, marking them Free, Blocked, or Unknown.

Representation

The grid can be represented as a 2-D array/matrix where 0 = free cell, 1 = obstacle, S = start, G = goal. The internal state of the agent is a similarly sized matrix that starts as 'Unknown' everywhere except cells the agent has already sensed, and is progressively filled in as the agent explores.

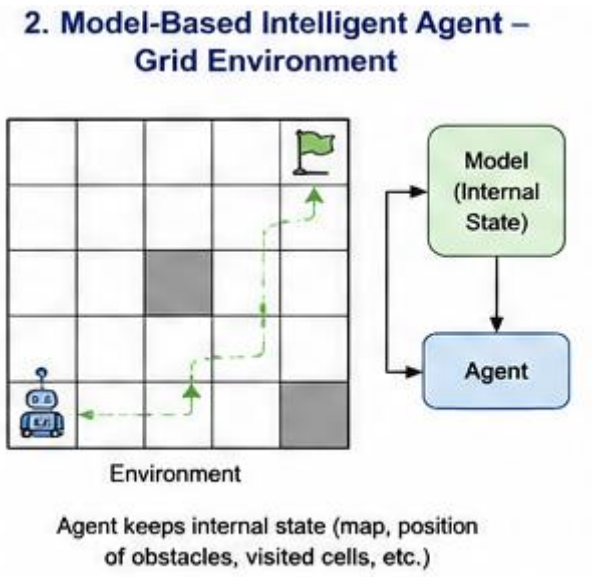
Algorithm

1. Initialize the grid environment and the agent's internal model (all cells Unknown).
2. Place the agent at the Start cell; sense the status of adjacent cells and update the internal model.
3. Repeat until the Goal is reached or no path remains:
   a. Using the internal model, choose the next move that makes progress toward the goal while avoiding known obstacles (e.g., a simple greedy Manhattan-distance rule, or BFS over the known-free cells).
   b. Execute the move; sense new precepts and update the internal model.
4. Report the path taken and whether the goal was reached.

Advantage over Simple Reflex Agent

Because the model-based agent remembers previously sensed obstacle information, it avoids revisiting known blocked cells and can plan more effectively even though the grid is only partially observable at any instant. This makes it strictly more capable than a simple reflex agent in environments that are not fully observable.

Diagram



7. Conclusion

A model-based intelligent agent was implemented for grid navigation with static obstacles. The agent successfully maintained and updated an internal state (world model) while navigating, and reached the goal cell while avoiding all obstacles, demonstrating the advantage of internal state over pure reactive behaviour.

8. Viva Voce Questions

1. What is the role of an internal state in a model-based agent?
2. Differentiate between a transition model and a sensor model.
3. Why is a model-based agent more suitable than a simple reflex agent for partially observable environments?
4. How would you represent the agent's internal world model in memory?
5. What is the difference between a model-based reflex agent and a goal-based agent?
6. How does the agent handle a dynamic environment where obstacles can move?
7. What is Manhattan distance and how is it used as a heuristic here?
8. What happens if the goal cell is completely enclosed by obstacles?

Assignment A3

Title

Implement A* Search for 8 puzzle problem.

1. Aim

To implement the A* search algorithm to solve the 8-puzzle problem using an appropriate heuristic function.

2. Prerequisite

State-space representation, concept of heuristic functions, data structures --priority queue/heap, basics of search in AI.

3. Objective

- To understand informed (heuristic) search techniques in Artificial Intelligence.
- To study the working of the A* search algorithm and its evaluation function f(n) = g(n) + h(n).
- To design and implement A* search to solve the 8-puzzle problem.
- To compare heuristic functions (Misplaced Tiles and Manhattan Distance) for efficiency.

4. Outcome

Students will be able to formulate the 8-puzzle as a state-space search problem and implement A* search using an admissible heuristic to find the optimal solution path from the initial to the goal configuration.

5. Software and Hardware Requirements

Python 3.x with heapq module / Java with Priority Queue / C++ with STL priority queue, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

The 8-Puzzle Problem

The 8-puzzle consists of a 3x3 board with 8 numbered tiles and one blank space. A tile adjacent to the blank space can slide into it. The objective is to reach a specified goal configuration from a given initial configuration using the minimum number of moves. It is a classic benchmark problem for search algorithms because its state space, though finite (9!/2 = 181,440 reachable states), is too large to search exhaustively for anything but small heuristics.

State-Space Formulation

State: an arrangement of the 8 tiles and the blank in the 3x3 grid.
Initial State: the given starting arrangement.
Actions: move blank Up, Down, Left, or Right (subject to boundary constraints).
Transition Model: swapping the blank with the adjacent tile in the chosen direction.
Goal Test: current state equals the goal arrangement.
Path Cost: number of moves made (each move costs 1).

A* Search Algorithm

A* is a best-first search algorithm that evaluates nodes using f(n) = g(n) + h(n), where g(n) is the exact cost from the start node to node n, and h(n) is a heuristic estimate of the cost from n to the nearest goal. A* is complete and, provided h(n) is admissible (never overestimates the true cost) and consistent, it is also optimal -- it is guaranteed to find the least-cost solution.

Heuristic Functions Used

h1 - Number of Misplaced Tiles: counts how many tiles are not in their goal position (blank excluded). It is admissible but a relatively weak heuristic.
h2 - Manhattan (City-Block) Distance: sums, over all tiles, the number of horizontal and vertical moves needed to bring each tile to its goal position. h2 is also admissible, dominates h1 (h2 >= h1 always), and typically expands far fewer nodes, making the search more efficient.

Data Structures

OPEN list: a priority queue (min-heap) ordered by f(n), holding nodes yet to be expanded.
CLOSED list: a set/hash table of already-expanded states, used to avoid re-expansion.
Each node stores: the board configuration, g(n), h(n), f(n), and a pointer to its parent (to reconstruct the solution path).

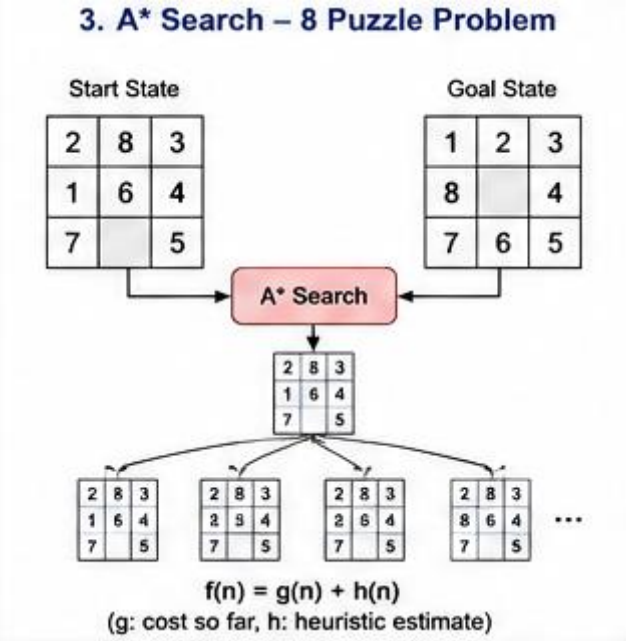
Algorithm Steps

1. Push the initial state into OPEN with g=0, h=h(initial), f=g+h.
2. While OPEN is not empty:
   a. Remove the node n with smallest f(n) from OPEN.
   b. If n is the goal, reconstruct and return the solution path.
   c. Move n to CLOSED.
   d. For each valid move (successor) of n: compute g, h, f; if not in CLOSED (or found with a better f), add/update it in OPEN.
3. If OPEN becomes empty without reaching the goal, report failure (unsolvable instance).

Solvability Check

Not every initial 8-puzzle configuration can reach the goal. A configuration is solvable if and only if the number of inversions (pairs of tiles that are out of their relative order, ignoring the blank) is even. This check should be performed before running A* to avoid an unbounded search on an unsolvable instance.

Diagram



7. Conclusion

A* search was successfully implemented to solve the 8-puzzle problem. Using the Manhattan Distance heuristic, the algorithm found the optimal (shortest) solution path while expanding significantly fewer nodes than with the Misplaced Tiles heuristic, confirming the effect of heuristic strength on search efficiency.

8. Viva Voce Questions

1. What is an admissible heuristic? Give an example for the 8-puzzle.
2. Why is A* both complete and optimal when h(n) is admissible?
3. Compare Misplaced Tiles and Manhattan Distance heuristics.
4. What is the difference between g(n), h(n), and f(n)?
5. What is a consistent (monotonic) heuristic? How does it relate to admissibility?
6. How do you check whether a given 8-puzzle instance is solvable?
7. What are the time and space complexities of A* search?
8. How does A* differ from Greedy Best-First Search?

Assignment A4

Title

Implement Adversarial Search with Alpha-Beta pruning.

1. Aim

To implement adversarial search using the Minimax algorithm with Alpha-Beta pruning for efficient decision making in a game environment.

2. Prerequisite

Concepts of game trees, Minimax algorithm, recursion, basics of two-player zero-sum games.

3. Objective

- To understand adversarial search and the Minimax decision rule for two-player games.
- To study how Alpha-Beta pruning improves upon plain Minimax by eliminating unnecessary branches.
- To implement Alpha-Beta pruning for a two-player zero-sum game (e.g., Tic-Tac-Toe).
- To analyse the reduction in the number of nodes explored compared to plain Minimax.

4. Outcome

Students will be able to model a two-player zero-sum game as a game tree and implement the Alpha-Beta pruning algorithm to select optimal moves while reducing search effort compared to plain Minimax.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Adversarial Search

In multi-agent competitive environments, one agent's gain is another's loss -- these are called zero-sum games. Search in such environments is adversarial because the opponent actively tries to minimize the agent's outcome. Classic examples include Tic-Tac-Toe, Chess, and Checkers.

Game Trees and Minimax

A game tree represents all possible moves from the current state. Nodes alternate between MAX (the player trying to maximize the utility) and MIN (the opponent trying to minimize it). The Minimax algorithm computes the Minimax value of every node by recursively backing up values from the leaves: at a MAX node it takes the maximum of the children's values, and at a MIN node it takes the minimum. The root value indicates the best achievable outcome for MAX, assuming optimal play by MIN.

Limitation of Plain Minimax

Plain Minimax explores the entire game tree, which is computationally expensive for large games (e.g., Chess has a branching factor of about 35). Its time complexity is O(b^m) where b is the branching factor and m is the maximum depth.

Alpha-Beta Pruning

Alpha-Beta pruning is an optimization of Minimax that prunes (skips) branches of the game tree that cannot possibly influence the final decision, without changing the result. It maintains two values while traversing the tree:
Alpha (alpha): the best (highest) value found so far for MAX along the current path.
Beta (beta): the best (lowest) value found so far for MIN along the current path.
A branch is pruned whenever alpha >= beta at a node, because the opponent (or the player) would never allow play to reach that branch.

Pruning Rules

At a MAX node: update alpha = max(alpha, value); if alpha >= beta, prune (cut off) the remaining children (beta cut-off).
At a MIN node: update beta = min(beta, value); if alpha >= beta, prune the remaining children (alpha cut-off).

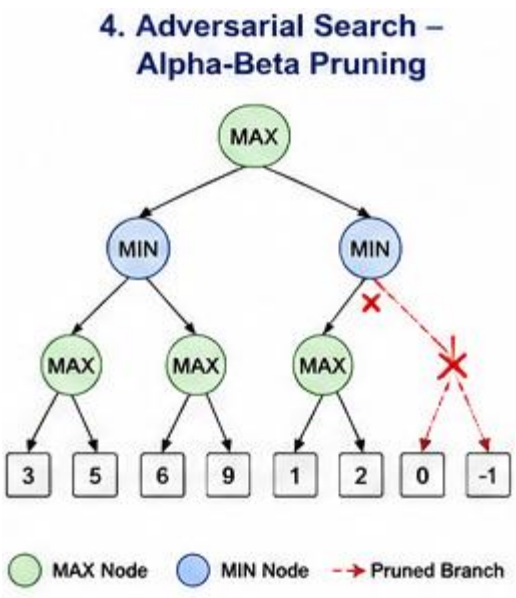
Efficiency

In the best case (when moves are examined in an optimal order), Alpha-Beta pruning reduces the effective branching factor from b to about sqrt(b), which doubles the search depth achievable in the same amount of time compared to plain Minimax, while always returning the same move that plain Minimax would.

Algorithm (Pseudocode Outline)

```
function ALPHABETA(node, depth, alpha, beta, maximizingPlayer):
  if depth == 0 or node is terminal: return utility(node)
  if maximizingPlayer:
    value = -infinity
    for each child of node:
      value = max(value, ALPHABETA(child, depth-1, alpha, beta, False))
      alpha = max(alpha, value)
      if alpha >= beta: break   // beta cut-off
    return value
  else:
    value = +infinity
    for each child of node:
      value = min(value, ALPHABETA(child, depth-1, alpha, beta, True))
      beta = min(beta, value)
      if alpha >= beta: break   // alpha cut-off
    return value
```

Diagram



7. Conclusion

Alpha-Beta pruning was implemented for a two-player zero-sum game and was found to select the same optimal move as plain Minimax while exploring substantially fewer nodes, demonstrating the effectiveness of pruning in adversarial search.

8. Viva Voce Questions

1. What is the difference between Minimax and Alpha-Beta pruning?
2. Define Alpha and Beta in the context of pruning.
3. Under what condition does pruning occur at a MAX node? At a MIN node?
4. Does Alpha-Beta pruning change the result compared to plain Minimax? Justify.
5. What is the best-case time complexity of Alpha-Beta pruning?
6. How does move ordering affect the amount of pruning achieved?
7. What is a zero-sum game? Give two real-world/game examples.
8. How would you extend Alpha-Beta pruning to games with more than two players?

Assignment A5

Title

Implement Tower of Hanoi by State Space Search.

1. Aim

To solve the Tower of Hanoi problem using the State Space Search technique.

2. Prerequisite

Concepts of recursion, stacks, state-space representation of problems.

3. Objective

- To understand state-space representation of a classic combinatorial problem.
- To study recursive problem decomposition for the Tower of Hanoi puzzle.
- To implement a recursive state-space search solution for Tower of Hanoi with N disks.
- To analyse the time complexity and minimum number of moves required.

4. Outcome

Students will be able to represent the Tower of Hanoi problem as a state-space search problem, implement its recursive solution, and derive the minimum number of moves required for N disks.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Problem Description

The Tower of Hanoi consists of three pegs (Source, Auxiliary, Destination) and N disks of different sizes stacked on the Source peg in decreasing order of size (largest at the bottom). The objective is to move the entire stack to the Destination peg, obeying the rules: (1) only one disk may be moved at a time, (2) a disk can only be placed on top of a larger disk or on an empty peg, and (3) only the topmost disk of a peg can be moved.

State-Space Formulation

State: the distribution of all N disks across the three pegs (which peg each disk is currently on).
Initial State: all N disks on the Source peg.
Goal State: all N disks on the Destination peg, in the same relative order.
Actions: move the topmost disk of one peg to the top of another peg, provided the move rules are respected.
The state space forms a graph that, for N disks, is isomorphic to the Sierpinski triangle, with exactly 3^N states and a shortest solution path of length 2^N - 1.

Recursive Solution

The classic recursive strategy decomposes the problem: to move N disks from Source to Destination using Auxiliary:
1. Recursively move the top N-1 disks from Source to Auxiliary, using Destination as the helper peg.
2. Move the remaining largest disk (disk N) directly from Source to Destination.
3. Recursively move the N-1 disks from Auxiliary to Destination, using Source as the helper peg.
The base case is N=1: simply move the single disk directly from Source to Destination.

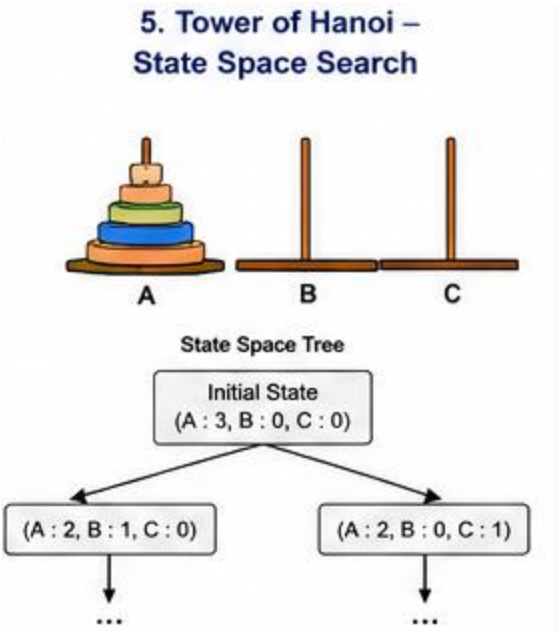
Minimum Number of Moves

Let T(N) be the minimum number of moves required for N disks. The recurrence is T(N) = 2*T(N-1) + 1, with T(1) = 1. Solving this recurrence gives T(N) = 2^N - 1. For example, T(3) = 7 and T(10) = 1023. This shows the problem has exponential time complexity in the number of disks.

State-Space Search View

Although Tower of Hanoi is usually solved by direct recursion, it can equally be framed and solved as a state-space search problem: each state is a configuration of disks on pegs, actions are legal single-disk moves, and search (e.g., BFS or IDDFS over this state graph) will also discover the same optimal 2^N - 1 length path, though direct recursion is far more efficient since it exploits the problem's self-similar structure instead of blind search.

Diagram



7. Conclusion

The Tower of Hanoi problem was solved using recursive state-space decomposition. The program correctly generated the sequence of moves for N disks and verified that the minimum number of moves matches the theoretical value of 2^N - 1.

8. Viva Voce Questions

1. State the three rules of the Tower of Hanoi puzzle.
2. Derive the recurrence relation for the minimum number of moves.
3. Why is the Tower of Hanoi problem said to have exponential time complexity?
4. How can Tower of Hanoi be modelled as a state-space search problem?
5. What is the size of the complete state space for N disks?
6. Can Tower of Hanoi be solved iteratively? Briefly describe one iterative approach.
7. What real-world problems have a similar recursive structure to Tower of Hanoi?
8. What would change in the algorithm if 4 pegs were allowed instead of 3?

Assignment A6

Title

Implement a Sudoku solver using backtracking and constraint propagation techniques.

1. Aim

To implement a Sudoku solver using backtracking and constraint propagation techniques.

2. Prerequisite

Concept of Constraint Satisfaction Problems (CSP), recursion/backtracking, 2-D arrays.

3. Objective

- To understand Constraint Satisfaction Problems (CSPs) and their formulation.
- To study backtracking search combined with constraint propagation for CSPs.
- To design and implement a Sudoku solver using backtracking with row/column/sub-grid constraint checking.
- To analyse how constraint propagation reduces the search space compared to plain backtracking.

4. Outcome

Students will be able to formulate Sudoku as a Constraint Satisfaction Problem and implement a backtracking algorithm enhanced with constraint propagation to efficiently solve any valid Sudoku puzzle.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Constraint Satisfaction Problems (CSP)

A CSP consists of a set of variables, each with a domain of possible values, and a set of constraints specifying allowable combinations of values. The task is to find an assignment of values to all variables that satisfies every constraint. Sudoku is a classical example of a CSP.

Sudoku as a CSP

Variables: each of the 81 cells in a 9x9 grid.
Domain: integers 1 to 9 for each empty cell (a fixed clue cell has a domain of exactly one value).
Constraints (all-different constraints): (1) every row must contain the digits 1-9 without repetition, (2) every column must contain the digits 1-9 without repetition, (3) every 3x3 sub-grid must contain the digits 1-9 without repetition.

Backtracking Search

Backtracking is a depth-first search over partial assignments. At each step it selects an unassigned variable, tries a value from its domain that is consistent with the constraints given the current partial assignment, and recursively continues. If no consistent value can be found for a variable, the algorithm backtracks -- it undoes the most recent assignment and tries the next alternative value.

Constraint Propagation

Constraint propagation techniques reduce the search space before/during backtracking by eliminating values from a variable's domain that cannot possibly satisfy the constraints. Two commonly combined techniques:
Forward Checking: whenever a value is assigned to a cell, remove that value from the domains of all cells sharing its row, column, or sub-grid.
Naked Singles / Constraint Propagation: if forward checking reduces a cell's domain to a single value, that value can be assigned immediately without further search, which cascades and often solves large portions of the grid without any guessing.

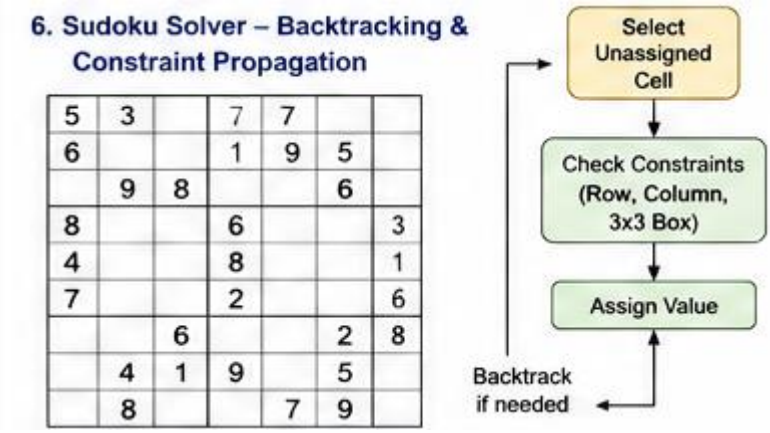
Variable and Value Ordering Heuristics

Minimum Remaining Values (MRV): choose the unassigned cell with the fewest legal values remaining -- this fails fast and prunes the search tree earlier.
Least Constraining Value: when trying values for a chosen cell, prefer the value that rules out the fewest choices for neighbouring cells.

Algorithm Steps

1. Represent the Sudoku grid as a 9x9 array; initialize with given clues.
2. Find the next empty cell (optionally using MRV).
3. For each candidate digit 1-9 consistent with row/column/sub-grid constraints:
   a. Place the digit (apply constraint propagation / forward checking).
   b. Recursively attempt to solve the rest of the grid.
   c. If the recursive call succeeds, return success.
   d. Otherwise, undo the digit (backtrack) and try the next candidate.
4. If no candidate works, return failure and backtrack to the previous cell.
5. If all cells are filled consistently, the puzzle is solved.

Diagram



7. Conclusion

A Sudoku solver was implemented using backtracking combined with constraint propagation (forward checking). The program correctly solved standard 9x9 Sudoku puzzles and constraint propagation was observed to substantially reduce the number of backtracking steps needed compared to plain brute-force backtracking.

8. Viva Voce Questions

1. Define a Constraint Satisfaction Problem (CSP) formally.
2. Formulate Sudoku in terms of variables, domains, and constraints.
3. What is forward checking and how does it speed up backtracking?
4. Explain the Minimum Remaining Values (MRV) heuristic.
5. What is arc consistency? How does it relate to Sudoku constraint propagation?
6. Why does plain backtracking without any propagation become slow for hard puzzles?
7. What is the difference between backtracking search and local search (e.g., hill climbing) for CSPs?
8. Give another real-world problem that can be modeled as a CSP.

Assignment A7

Title

Implement Local Beam Search for The Traveling Salesman Problem (TSP).

1. Aim

To implement the Local Beam Search algorithm to obtain an optimal or near-optimal solution for the Traveling Salesman Problem.

2. Prerequisite

Concept of local search, hill climbing, basics of optimization problems, graph representation of distances.

3. Objective

- To understand local search techniques as an alternative to systematic search for optimization problems.
- To study the Local Beam Search algorithm and how it differs from simple hill climbing.
- To implement Local Beam Search for solving the Traveling Salesman Problem (TSP).
- To analyze the quality of solutions obtained versus the beam width (k) used.

4. Outcome

Students will be able to apply Local Beam Search, a parallel local-search technique, to obtain near-optimal solutions to the Traveling Salesman Problem and understand the trade-off between beam width and solution quality.

5. Software and Hardware Requirements

Python 3.x with NumPy / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

The Traveling Salesman Problem (TSP)

Given a set of cities and the distances between each pair of cities, the TSP asks for the shortest possible route that visits every city exactly once and returns to the starting city. TSP is NP-hard: no known algorithm can solve all instances optimally in polynomial time, so for larger instances we rely on heuristic/local search methods that give good (not necessarily optimal) solutions quickly.

Local Search

Unlike systematic search, local search algorithms operate on a single current state (or a small set of states) and move to neighbouring states, without keeping track of paths or the set of unvisited states. They use very little memory and can find reasonable solutions in large or infinite state spaces where systematic search is infeasible. Hill Climbing is the simplest local search: it always moves to the best neighbour, but a single search state means it easily gets stuck at local maxima, ridges, or plateaus.

Local Beam Search

Local Beam Search keeps track of k states (the 'beam') rather than one. It begins with k randomly generated states. At every step, all the successors of all k states are generated. If any successor is the goal, the algorithm stops; otherwise it selects the k best successors from the complete list of successors and repeats. Unlike running k independent random restarts, Local Beam Search shares information between the k parallel searches -- if one state generates several good successors while the rest generate poor ones, all k selected states can concentrate around the promising region ('come here, the grass is greener'), which can also cause loss of diversity.

Applying Local Beam Search to TSP

State: a complete tour (a permutation of all cities, visited once, returning to start).
Objective/Cost function: total tour distance (sum of edge distances along the tour) -- we want to minimize this (equivalently, maximize the negative of the cost as the 'value').
Successor generation: apply a small perturbation such as a 2-opt swap (reverse a segment of the tour) or swapping two cities' positions, to generate neighbouring tours.
Beam width k: number of tours retained at each iteration -- larger k explores more of the space at the cost of more computation.

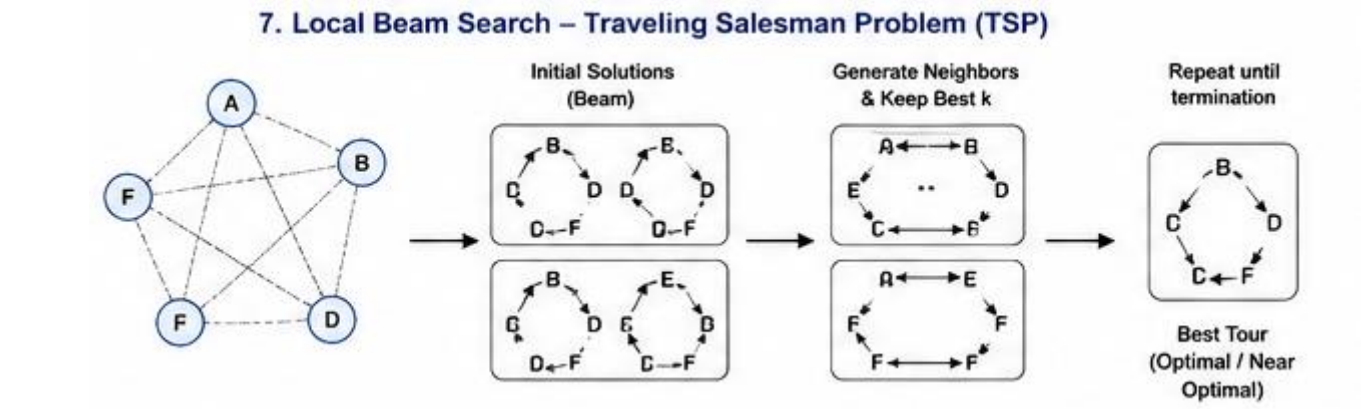
Algorithm Steps

1. Randomly generate k initial tours (the beam).
2. Repeat until a stopping criterion is met (max iterations / no improvement over several rounds):
   a. For each of the k tours, generate a set of neighbouring tours (e.g., via 2-opt swaps).
   b. Pool all generated neighbours together with the current beam.
   c. Compute the total distance (cost) of every tour in the pool.
   d. Select the k tours with the lowest cost as the new beam.
3. Return the best (shortest) tour found across all iterations.

Trade-offs

A small beam width (k) behaves closer to Stochastic Hill Climbing and risks getting stuck in a poor local optimum; a larger k explores more of the search space and generally yields shorter tours, but at higher computational and memory cost per iteration. Local Beam Search does not guarantee the globally optimal tour, but in practice yields good approximate solutions much faster than exhaustive search.

Diagram



7. Conclusion

Local Beam Search was implemented for the Traveling Salesman Problem. The algorithm produced near-optimal tours within a small number of iterations, and increasing the beam width k was observed to improve solution quality at the cost of additional computation, illustrating the exploration-exploitation trade-off in local search.

8. Viva Voce Questions

1. What is the key difference between Hill Climbing and Local Beam Search?
2. Why is TSP considered an NP-hard problem?
3. What is a 2-opt move and why is it used as a successor function for TSP?
4. How does the beam width k affect solution quality and computation time?
5. What is a local optimum, and how can Local Beam Search still get trapped in one?
6. How is Local Beam Search different from running k independent Hill Climbing searches (random-restart)?

Assignment B1

Title

Use BFS to solve a planning problem. Examples can be 8 puzzle problem, Robot path planning, Blocks world etc.

1. Aim

To implement Breadth-First Search (BFS) to solve a planning or problem-solving task such as the 8-puzzle, robot path planning, or Blocks World problem.

2. Prerequisite

State-space representation, queue data structure, basics of graph traversal.

3. Objective

- To understand uninformed (blind) search strategies in Artificial Intelligence.
- To study the Breadth First Search (BFS) algorithm and its properties.
- To implement BFS to solve a chosen planning problem (8-puzzle, robot path planning, or Blocks World).
- To analyse the completeness, optimality, time and space complexity of BFS.

4. Outcome

Students will be able to formulate a planning problem as a state-space graph and implement BFS to find the shortest (least number of moves) solution path from the initial state to the goal state.

5. Software and Hardware Requirements

Python 3.x with collections.deque / Java with LinkedList as Queue / C++ with STL queue, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Uninformed Search

Uninformed (blind) search strategies have no additional information about states beyond that provided in the problem definition -- they can only generate successors and test whether a state is the goal. Breadth First Search (BFS) is one of the simplest uninformed strategies.

Breadth First Search (BFS)

BFS expands the shallowest (least-deep) unexpanded node first. It is implemented using a FIFO queue: newly generated successors go to the back of the queue, so shallower nodes are always expanded before deeper ones. BFS explores the search tree level by level.

Properties of BFS

Completeness: BFS is complete -- if a solution exists, it will be found, provided the branching factor is finite.
Optimality: BFS is optimal (finds the shallowest / least-cost solution) provided all step costs are equal (e.g., 1 per move).
Time Complexity: O(b^d), where b is the branching factor and d is the depth of the shallowest goal.
Space Complexity: O(b^d) -- BFS keeps every generated node in memory, which is its major drawback for large search spaces.

Example Application: Robot Path Planning

In grid-based robot path planning, each grid cell is a state, and actions move the robot to an adjacent free cell (Up/Down/Left/Right). BFS explores the grid outward from the start cell layer by layer and is guaranteed to find the shortest path (fewest cell-to-cell moves) to the goal, since every move has equal cost.

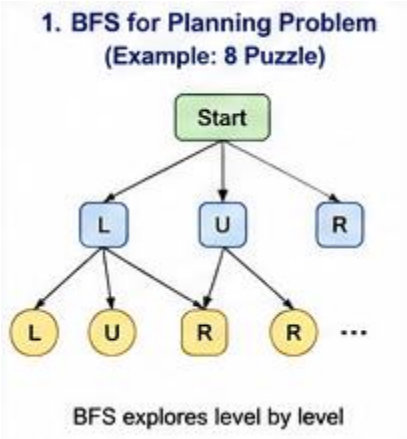
Example Application: 8-Puzzle

Each board configuration is a state; actions slide the blank Up/Down/Left/Right. BFS explores all configurations reachable in 1 move, then 2 moves, and so on, guaranteeing the shortest solution -- however BFS's memory requirement grows extremely fast for the 8-puzzle since the branching factor is up to 4 and solutions can be many moves deep, so BFS is practical only for puzzles with short solutions.

Algorithm Steps

1. Create a FIFO queue and enqueue the start state; mark it visited.
2. While the queue is not empty:
   a. Dequeue the front state n.
   b. If n is the goal, reconstruct the path (using parent pointers) and return it.
   c. Otherwise, generate all valid successors of n; enqueue each unvisited successor and mark it visited.
3. If the queue empties without finding the goal, report failure (no solution exists).

Diagram



7. Conclusion

BFS was implemented and applied to solve a chosen planning problem. The algorithm correctly found the shortest solution path (in terms of number of moves), and its completeness and optimality (for unit-cost problems) were verified experimentally on multiple test instances.

8. Viva Voce Questions

1. Why is BFS said to be optimal only when all step costs are equal?
2. What data structure is central to BFS, and why?
3. Give the time and space complexity of BFS in terms of b and d.
4. Compare BFS with Depth First Search in terms of memory usage.
5. Under what circumstances would BFS run out of memory in practice?
6. How would you modify BFS to handle unequal (weighted) step costs?
7. What is the difference between graph-search and tree-search versions of BFS?
8. Give the PEAS description for robot path planning if used as your chosen application.

Assignment B2

Title

Use DFS for problem solving. Examples can be water jug problem , Missionaries and Cannibals ,maze solving, blocks world etc.

1. Aim

To implement Depth-First Search (DFS) to solve a problem-solving task such as the Water Jug problem, Missionaries and Cannibals problem, maze solving, or Blocks World problem.

2. Prerequisite

State-space representation, stack/recursion, basics of graph traversal.

3. Objective

- To understand the Depth First Search (DFS) strategy and its trade-offs versus BFS.
- To formulate a classic AI puzzle (Water Jug / Missionaries and Cannibals / Maze) as a state-space search problem.
- To implement DFS to find a solution path for the chosen problem.
- To analyze completeness, optimality, time and space complexity of DFS.

4. Outcome

Students will be able to formulate a classic search puzzle as a state-space problem and implement DFS to explore the state space and find a valid solution path.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler (stack via recursion or explicit stack), Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Depth First Search (DFS)

DFS always expands the deepest unexpanded node in the current search tree first. It uses a LIFO stack (or equivalently, recursion) for the frontier: the most recently generated node is expanded next. When a branch reaches a dead end (no more unexplored successors), the search backtracks to the shallowest unexpanded node.

Properties of DFS

Completeness: DFS is not complete in infinite state spaces or graphs with cycles (unless cycle/repeated-state checking is used) -- it can go down an infinite branch and never find the solution.
Optimality: DFS is not optimal -- it may return a longer path even though a shorter one exists, because it stops at the first goal found regardless of depth.
Time Complexity: O(b^m), where m is the maximum depth of the search tree (which may be much larger than the solution depth).
Space Complexity: O(b*m) -- DFS only needs to store the nodes on the current path plus their unexpanded siblings, which is a major memory advantage over BFS.

Example: Water Jug Problem

Given two jugs of known capacities (e.g., 4 litres and 3 litres) and no measuring marks, the goal is to measure out a target quantity (e.g., 2 litres) using operations: fill a jug completely, empty a jug completely, or pour from one jug into the other until either the source is empty or the destination is full. Each state is the pair (amount in jug 1, amount in jug 2); DFS explores this state graph looking for a state where one jug holds the target amount.

Example: Missionaries and Cannibals

Three missionaries and three cannibals must cross a river using a boat that holds at most two people, such that cannibals never outnumber missionaries on either bank. A state is (#missionaries, #cannibals, boat position) on the starting bank. DFS explores possible boat crossings, backtracking whenever an unsafe state (cannibals > missionaries on a bank, with missionaries present) is reached.

Example: Maze Solving

A maze is represented as a 2-D grid with walls and open paths. DFS explores one path as deep as possible, backtracking upon hitting a dead end, until the exit is found. This mirrors the natural 'follow-the-wall' exploration strategy used in physical maze solving.

Algorithm Steps

1. Push the start state onto a stack (or begin recursion); mark it visited.
2. While the stack is not empty:
   a. Pop the top state n.
   b. If n is the goal, reconstruct and return the solution path.
   c. Otherwise, generate valid, unvisited successors of n; push them onto the stack and mark visited.
3. If the stack empties without finding the goal, report failure.

Diagram



7. Conclusion

DFS was implemented and used to solve the chosen classic AI puzzle. The algorithm successfully found a valid solution path by exploring the state space depth-first with backtracking, and its memory efficiency compared to BFS was observed on the given problem instances.

8. Viva Voce Questions

1. Why is DFS not guaranteed to be optimal?
2. How does DFS achieve better space complexity than BFS?
3. What problems can arise if DFS is applied to a graph with cycles, and how is it fixed?
4. Formulate the Water Jug problem's states, actions, and goal test.
5. What safety constraint must hold at every state of the Missionaries and Cannibals problem?
6. Compare Depth-Limited Search and Iterative Deepening Search with plain DFS.
7. In what kind of problems would you prefer DFS over BFS, and why?
8. How would you avoid revisiting the same state repeatedly in DFS (cycle checking)?

Assignment B3

Title

Implement Backtracking search to solve the 8-Queens constraint satisfaction problem.

1. Aim

To implement a backtracking search algorithm to solve the 8-Queens Constraint Satisfaction Problem.

2. Prerequisite

Concept of Constraint Satisfaction Problems, recursion, 2-D board representation.

3. Objective

- To understand the N-Queens problem as a Constraint Satisfaction Problem (CSP).
- To study the backtracking search technique for solving CSPs.
- To implement backtracking search to solve the 8-Queens problem.
- To analyze pruning strategies that improve backtracking efficiency for N-Queens.

4. Outcome

Students will be able to formulate the 8-Queens problem as a CSP and implement backtracking search to find one or all valid solutions where no two queens attack each other.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

The 8-Queens Problem

The 8-Queens problem asks: how can 8 queens be placed on an 8x8 chessboard so that no two queens attack each other? Two queens attack each other if they share the same row, column, or diagonal. The problem generalizes to N-Queens on an NxN board.

CSP Formulation

Variables: one variable per column, Q1..Q8, representing the row position of the queen placed in that column (this formulation, by construction, already avoids column conflicts).
Domain: {1, 2, ..., 8} -- the possible row for each queen.
Constraints: for every pair of columns i != j: Qi != Qj (no same row) and |Qi - Qj| != |i - j| (no same diagonal).

Backtracking Search

Backtracking places queens column by column. For each column, it tries every row in the domain and checks whether the placement is 'safe' -- consistent with all previously placed queens (no same row, no same diagonal). If a safe row is found, the queen is placed and the search proceeds to the next column. If no row in the current column is safe, the algorithm backtracks to the previous column and tries its next alternative row.

Safety Check

A placement of a queen at (row, col) is safe with respect to already-placed queens if, for every previously placed queen at (r, c): row != r (different row) and abs(row - r) != abs(col - c) (not on the same diagonal). Since queens are placed one per column, same-column conflicts are avoided automatically by construction.

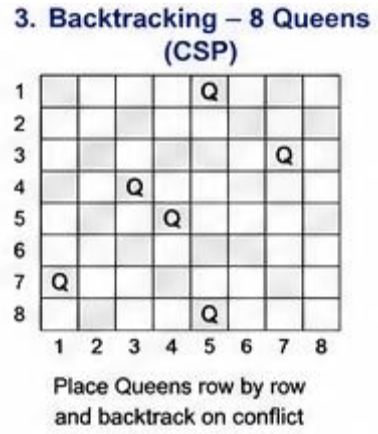
Algorithm Steps

1. Start at column 0 with an empty board.
2. If all N queens have been placed (column == N), record/return the current solution.
3. Otherwise, for row = 0 to N-1:
   a. If placing a queen at (row, column) is safe, place it.
   b. Recursively attempt to solve for column+1.
   c. If the recursive call succeeds (a full solution was found) and only one solution is required, return success.
   d. Otherwise, remove the queen from (row, column) -- backtrack -- and try the next row.
4. If no row works for this column, return failure to the caller (backtrack further).

Number of Solutions

For N=8, backtracking finds 92 distinct solutions (12 of which are 'fundamental' -- the rest are rotations/reflections of these 12). Backtracking with the safety check dramatically reduces the search compared to a brute-force check of all 8^8 (over 16 million) row assignments.

Diagram



7. Conclusion

Backtracking search was implemented for the 8-Queens problem. The program correctly found valid queen placements satisfying all row, column and diagonal constraints, and was extended to enumerate all 92 solutions, confirming the correctness and effectiveness of constraint-based backtracking.

8. Viva Voce Questions

1. Formulate the N-Queens problem as a CSP (variables, domains, constraints).
2. How many total solutions exist for the standard 8-Queens problem?
3. What check determines whether a queen placement is 'safe'?
4. How does representing one queen per column avoid the need for a row/column check?
5. How does backtracking prune the search tree compared to generating all N^N placements?
6. What is the difference between backtracking and brute-force search?
7. How would you modify the algorithm to find all solutions instead of just one?
8. How does the MRV (Minimum Remaining Values) heuristic apply to N-Queens?

Assignment B4

Title

Develop a simple chatbot using predefined rules and pattern matching.

1. Aim

To develop a simple chatbot using predefined rules and pattern-matching techniques.

2. Prerequisite

Basics of string processing/regular expressions, dictionaries/hash maps, elementary NLP concepts.

3. Objective

- To understand the architecture of a rule-based conversational agent (chatbot).
- To study pattern matching techniques used to map user input to appropriate responses.
- To design and implement a simple chatbot using a predefined set of pattern-response rules.
- To analyze the limitations of rule-based chatbots compared to modern AI-based conversational systems.

4. Outcome

Students will be able to design a rule base of patterns and responses and implement a chatbot that processes natural-language user input, matches it against these rules, and produces a coherent reply, including a fallback for unmatched input.

5. Software and Hardware Requirements

Python 3.x with re (regular expressions) module / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

What is a Chatbot?

A chatbot is a software application designed to simulate human conversation through text (or voice). Chatbots range from simple rule-based systems that match patterns in user input to sophisticated AI-based systems using large language models. This assignment focuses on the simplest category: a rule-based, pattern-matching chatbot, historically exemplified by Joseph Weizenbaum's ELIZA (1966).

Rule-Based Pattern Matching

A rule-based chatbot maintains a knowledge base of pattern-response pairs. Each pattern is typically expressed using keywords or regular expressions that can match a family of related user inputs (e.g., a pattern for 'I am *' can capture 'I am tired', 'I am happy', etc., and reflect the captured phrase back in the response, e.g., 'Why are you * ?').

Processing Pipeline

1. Pre-processing: convert user input to lowercase, strip punctuation, and tokenize into words.
2. Pattern Matching: compare the pre-processed input (or its tokens) against the rule base, typically using regular expressions or keyword lookup, often trying rules in a priority order (more specific rules first).
3. Response Generation: when a rule matches, generate the response, substituting any captured groups from the input (as ELIZA-style reflection) if applicable.
4. Fallback: if no rule matches, return a generic response (e.g., 'Can you tell me more about that?') so the conversation does not break down.

Example Rule Base (Illustrative)

Pattern (keyword / regex)	Response Template
hello|hi|hey	Hello! How can I help you today?
my name is (.*)	Nice to meet you, \1!
i am (.*)	Why do you think you are \1?
bye|quit|exit	Goodbye! Have a great day.
(default / no match)	I'm not sure I understand -- could you rephrase that?

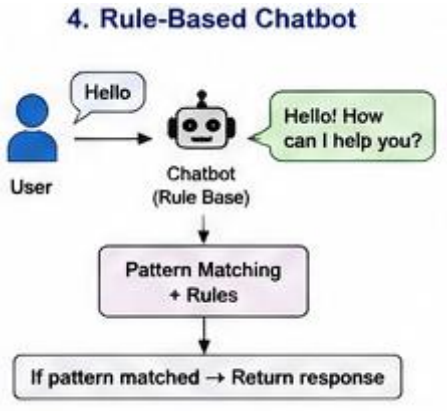
Limitations of Rule-Based Chatbots

Rule-based chatbots can only respond well to inputs anticipated by their designer; they do not truly 'understand' language, cannot handle context across multiple turns beyond simple slot-filling, and their quality depends entirely on the comprehensiveness of the hand-written rule base. Modern chatbots instead use statistical/neural NLP models (e.g., transformer-based large language models) trained on large corpora, allowing them to generalize to unseen inputs far better than fixed pattern-matching.

Algorithm Steps

1. Load the predefined pattern-response rule base.
2. Loop: read user input.
3. Pre-process the input (lowercase, strip punctuation).
4. Try to match input against each rule's pattern in priority order.
5. If matched, format and display the response (substituting captured text if needed).
6. If not matched, display the fallback response.
7. Exit the loop when the user enters an exit keyword (e.g., 'bye').

Diagram



7. Conclusion

A rule-based chatbot was implemented using a predefined set of pattern-response rules and regular-expression matching. The chatbot correctly handled greetings, simple statements, and farewells, and gracefully used a fallback response for unrecognized input, illustrating both the utility and the limitations of pure pattern-matching conversational agents.

8. Viva Voce Questions

1. What is the fundamental technique used by a rule-based chatbot to generate responses?
2. How does ELIZA's 'reflection' technique work?
3. What are the main limitations of rule-based chatbots compared to AI/ML-based chatbots?
4. Why is a fallback response important in a chatbot's design?
5. How would you use regular expressions to capture and reuse part of the user's input?
6. What is the difference between rule-based and retrieval-based versus generative chatbots?
7. How could you extend this chatbot to maintain context across multiple turns?
8. What NLP pre-processing steps improve pattern-matching accuracy?

Assignment B5

Title

. Design an intelligent agent that guesses a number using feedback (higher/lower).

1. Aim

To design and implement an intelligent agent that guesses a number based on higher and lower feedback provided by the user

2. Prerequisite

Basics of searching/sorting, binary search concept, simple I/O programming.

3. Objective

- To understand how an agent can use feedback from the environment to refine its search.
- To study the Binary Search strategy as applied to a number-guessing agent.
- To design and implement an intelligent agent that guesses a secret number using higher/lower feedback.
- To analyze the number of guesses required and relate it to logarithmic time complexity.

4. Outcome

Students will be able to design an intelligent agent that uses percept feedback (higher/lower/correct) to iteratively narrow its search range and correctly guess a target number in the minimum expected number of attempts.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Problem Description

The environment picks (or the user thinks of) a secret number within a known range [low, high]. The agent's task is to guess the number in as few attempts as possible. After each guess, the environment provides one of three feedback percepts: 'Higher' (the secret number is greater than the guess), 'Lower' (the secret number is less than the guess), or 'Correct' (the guess matches the secret number, ending the episode).

Agent Design -- Binary Search Strategy

The agent behaves as a goal-based/utility-based agent: given the percept (feedback), it updates its belief about the possible range containing the secret number, and always guesses the midpoint of that range next. This binary search strategy is provably optimal in the worst case among all possible guessing strategies for this task, because every guess-feedback pair yields exactly one bit of information (higher / lower), and log2(N) bits are needed to identify one number among N possibilities.

Working Cycle of the Agent

1. Initialize the internal state: low = range minimum, high = range maximum.
2. Compute guess = floor((low + high) / 2) and present it as the action.
3. Perceive feedback from the environment:
   - If feedback == 'Correct': terminate; number of guesses used is the performance measure (lower is better).
   - If feedback == 'Higher': update low = guess + 1 (secret number lies in the upper half).
   - If feedback == 'Lower': update high = guess - 1 (secret number lies in the lower half).
4. Repeat from step 2 until the number is guessed.

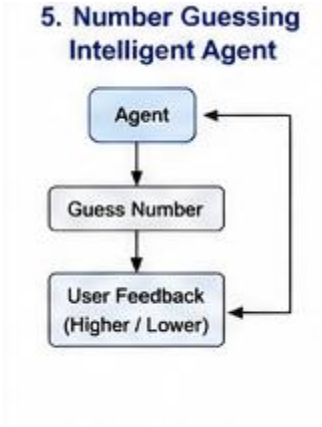
Complexity Analysis

Since each feedback halves the remaining search range, the worst-case number of guesses needed to identify a number among N possibilities is ceil(log2(N)) + 1 -- for example, at most 7 guesses for a range of 1 to 100. This matches the well-known logarithmic time complexity of binary search, O(log N).

Rationality of the Agent

The agent is rational with respect to the performance measure 'minimize number of guesses', because at every step it selects the action (guess) that provably minimizes the expected/worst-case number of remaining guesses, given the percept sequence received so far (the sequence of higher/lower feedbacks).

Diagram



7. Conclusion

An intelligent number-guessing agent was implemented using a binary-search-based strategy driven by higher/lower feedback. The agent consistently guessed the secret number within ceil(log2(N)) + 1 attempts across multiple test ranges, confirming the optimality of the binary search approach for this feedback-based guessing task.

8. Viva Voce Questions

1. Why is binary search the optimal strategy for this number-guessing task?
2. What is the worst-case number of guesses for a range of 1 to 1000?
3. What type of agent (reflex/model-based/goal-based/utility-based) best describes this agent, and why?
4. How much information (in bits) does each higher/lower feedback provide?
5. How would the strategy change if the feedback were noisy (sometimes incorrect)?
6. What is the difference between a deterministic and a stochastic environment, and which applies here?
7. How would you adapt the agent if guesses had different costs depending on their value?
8. Relate this agent's strategy to the general concept of an information gain heuristic.

Assignment B6

Title

Implement Goal Stack Planning for solving the Blocks World problem and achieve a specified goal configuration.

1. Aim

To implement Goal Stack Planning for solving the Blocks World problem and achieving a specified goal configuration.

2. Prerequisite

Concepts of STRIPS-style operators (preconditions/add/delete lists), stacks, planning basics.

3. Objective

- To understand classical planning and the STRIPS representation of actions.
- To study the Goal Stack Planning algorithm for generating a plan of operators.
- To implement Goal Stack Planning to solve a Blocks World problem and reach a specified goal configuration.
- To analyse how goal stack planning handles conjunctive goals and their interactions.

4. Outcome

Students will be able to represent Blocks World actions in STRIPS form (preconditions, add-list, delete-list) and implement Goal Stack Planning to produce a correct, ordered sequence of operators transforming an initial block configuration into a specified goal configuration.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

The Blocks World

Blocks World is a classical AI planning domain consisting of a set of labelled blocks (e.g., A, B, C) that can be stacked on top of one another or placed on a table, and a robot arm that can move one block at a time. It is small enough to reason about completely, yet rich enough to demonstrate the core ideas of classical planning, including goal interaction.

STRIPS Representation

STRIPS (STanford Research Institute Problem Solver) represents each action/operator using three components:
Preconditions: facts that must be true before the action can be executed.
Add List: facts that become true after the action is executed.
Delete List: facts that become false (no longer hold) after the action is executed.
Typical Blocks World predicates: On(x, y), OnTable(x), Clear(x), ArmEmpty, Holding(x).

Example Operators

Operator	Preconditions	Add List	Delete List
Stack(x, y)	Holding(x), Clear(y)	On(x,y), Clear(x), ArmEmpty	Holding(x), Clear(y)
Unstack(x, y)	On(x,y), Clear(x), ArmEmpty	Holding(x), Clear(y)	On(x,y), Clear(x), ArmEmpty
PickUp(x)	OnTable(x), Clear(x), ArmEmpty	Holding(x)	OnTable(x), Clear(x), ArmEmpty
PutDown(x)	Holding(x)	OnTable(x), Clear(x), ArmEmpty	Holding(x)

Goal Stack Planning Algorithm

Goal Stack Planning maintains a single stack that interleaves goals (predicates to be achieved) and operators (once all their preconditions are satisfied). The algorithm processes the stack as follows: push the overall goal (which may be a conjunction of sub-goals) onto the stack, then push the current world-state description beneath it. Repeatedly examine the top of the stack:

- If it is a goal already true in the current state, pop it (nothing to do).
- If it is a goal not yet true, push an operator that can achieve it, then push that operator's unsatisfied preconditions on top (as new sub-goals).
- If it is an operator whose preconditions are all satisfied, pop and apply it -- update the current state, and append the operator to the growing plan.
- If the popped item is a state marker, replace the 'current state' record with it.

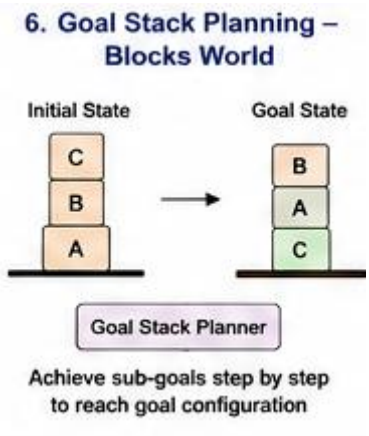
Handling Conjunctive Goals and Interactions

When the goal consists of multiple conditions (e.g., On(A,B) AND On(B,C)), goal stack planning pushes each conjunct as a separate sub-goal. A known limitation is the Sussman Anomaly: achieving one sub-goal can undo a previously achieved sub-goal because goal stack planning processes goals independently without considering their interaction, sometimes requiring the goals to be reordered or the plan to be revisited to reach a correct solution.

Algorithm Steps

1. Push the goal state (conjunction of goal predicates) onto the Goal Stack.
2. Push the initial state description as the 'current state' marker beneath the goals.
3. While the Goal Stack is not empty:
   a. Pop the top item.
   b. If it is a single goal predicate true in the current state, discard it (continue).
   c. If it is a single goal predicate not true: choose an operator whose add-list contains it; push the operator, then push its unsatisfied preconditions as sub-goals.
   d. If it is an operator with all preconditions satisfied: apply it (update current state using its add/delete lists) and append it to the Plan.
4. Return the Plan -- the ordered sequence of applied operators.

Diagram



7. Conclusion

Goal Stack Planning was implemented for the Blocks World domain using STRIPS-style operators. The algorithm successfully generated a valid, ordered plan of Stack/Unstack/PickUp/PutDown operations transforming a given initial block configuration into the specified goal configuration.

8. Viva Voce Questions

1. What are the three components of a STRIPS operator?
2. Explain the Add List and Delete List with an example from Blocks World.
3. What is the Sussman Anomaly, and why does it challenge goal stack planning?
4. How does Goal Stack Planning differ from Partial-Order Planning?
5. What predicates are typically used to represent a Blocks World state?
6. Why is a single stack (rather than a queue) appropriate for this planning technique?
7. How would you detect that a goal is already satisfied in the current state?
8. What is the difference between classical planning and search-based problem solving?

Assignment B7

Title

Develop a game-playing agent using the Minimax algorithm for optimal decision-making.

1. Aim

To develop a game-playing intelligent agent using the Minimax algorithm for optimal decision-making.

2. Prerequisite

Concepts of game trees, recursion, two-player zero-sum games, utility functions.

3. Objective

- To understand the Minimax decision procedure for optimal play in deterministic, two-player, zero-sum games.
- To study how a static evaluation function is used at cut-off depth when the full game tree cannot be explored.
- To implement a game-playing agent using the Minimax algorithm for a game such as Tic-Tac-Toe.
- To analyze the optimality and complexity of Minimax-based decision making.

4. Outcome

Students will be able to implement the Minimax algorithm to build a game-playing agent capable of choosing an optimal move at every turn, assuming rational play by the opponent.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Two-Player Zero-Sum Games

In a deterministic, fully observable, two-player, zero-sum game (such as Tic-Tac-Toe, Chess, or Checkers), the two players -- conventionally called MAX and MIN -- alternate moves, and one player's gain is exactly the other's loss. Game playing is treated as a search problem defined by: the initial state, a function returning the legal moves in a state, a transition (result) model, a terminal test, and a utility function assigning a numeric value to terminal states (e.g., +1 for a MAX win, -1 for a MIN win, 0 for a draw).

The Minimax Algorithm

Minimax computes the optimal move for MAX by assuming that MIN always plays optimally (to minimize MAX's outcome). It works by recursively computing the Minimax value of every node in the game tree:
Minimax(terminal node) = Utility(node).
Minimax(MAX node) = maximum of Minimax(children).
Minimax(MIN node) = minimum of Minimax(children).
MAX then picks the move leading to the child with the highest Minimax value.

Static (Heuristic) Evaluation Function

For games with a game tree too large to search to termination (like Chess), Minimax is combined with a depth cut-off and a static evaluation function Eval(s) that estimates the utility of non-terminal state s (e.g., in Tic-Tac-Toe, a simple evaluation could count the number of possible winning lines still open for MAX minus those open for MIN). Deeper search generally produces better play, subject to available time and the quality of the evaluation function.

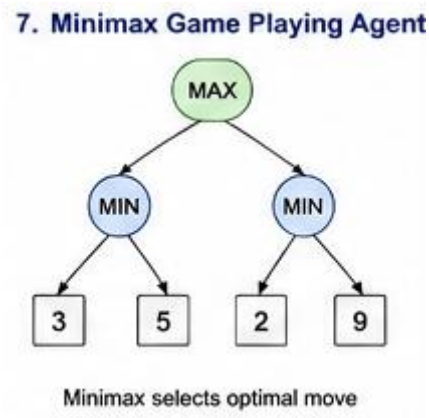
Applying Minimax to Tic-Tac-Toe

State: the 3x3 board configuration.
Terminal Test: a player has three in a row/column/diagonal, or the board is full (draw).
Utility Function: +1 if MAX (say, 'X') wins, -1 if MIN ('O') wins, 0 for a draw.
Because Tic-Tac-Toe's entire game tree has at most 9! = 362,880 leaf states, Minimax can search the tree exhaustively to the terminal states without needing a cut-off or heuristic evaluation -- guaranteeing perfect play (an agent using full Minimax on Tic-Tac-Toe can never lose).

Algorithm Steps

1. Given the current board and whose turn it is (MAX or MIN), check if it is a terminal state; if so return its utility.
2. If MAX's turn: initialize best = -infinity; for each legal move, recursively compute Minimax of the resulting state (as MIN's turn); best = max(best, that value). Return best.
3. If MIN's turn: initialize best = +infinity; for each legal move, recursively compute Minimax of the resulting state (as MAX's turn); best = min(best, that value). Return best.
4. At the root, choose the move whose resulting Minimax value equals the computed best value -- this is the agent's optimal move.

Diagram



7. Conclusion

A game-playing agent for Tic-Tac-Toe was implemented using the Minimax algorithm. The agent was verified to always play optimally -- it never lost a game against any opponent strategy, and it consistently achieved a win whenever the opponent made a sub-optimal move, confirming correct Minimax behaviour.

8. Viva Voce Questions

1. State the Minimax decision rule for MAX and MIN nodes.
2. Why does full Minimax guarantee perfect play in Tic-Tac-Toe?
3. What is a static evaluation function, and when is it needed?
4. What is the time complexity of plain Minimax in terms of branching factor b and depth m?
5. How does depth cut-off affect the quality of Minimax's decisions?
6. What assumption does Minimax make about the opponent's play?
7. How would you adapt Minimax for a game with more than two players?
8. How does Alpha-Beta pruning relate to and improve upon Minimax (refer to Assignment A4)?

Assignment C1

Title

Medical Diagnosis System (Rule-Based Expert System) using forward and backward chaining.

1. Aim

To develop a rule-based Medical Diagnosis Expert System using forward chaining and backward chaining techniques to assist in identifying possible diseases based on given symptoms.

2. Prerequisite

Concepts of propositional/predicate logic, IF-THEN rules, basics of knowledge representation, Assignment B6/B7 background on rule-based reasoning helpful.

3. Objective

- To understand the architecture of a rule-based expert system: knowledge base, working memory, and inference engine.
- To study Forward Chaining and Backward Chaining as inference strategies.
- To design and implement a simple Medical Diagnosis expert system with a rule base linking symptoms to diseases.
- To compare forward chaining and backward chaining in terms of applicability and efficiency for diagnosis.

4. Outcome

Students will be able to design a knowledge base of medical IF-THEN rules and implement both forward-chaining and backward-chaining inference engines to diagnose a probable disease from a given set of patient symptoms.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler (or a rule-engine library such as Python's `experta`), Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Expert Systems

An expert system is an AI program that emulates the decision-making ability of a human domain expert, typically in a narrow domain such as medical diagnosis, using a base of encoded expert knowledge and a reasoning (inference) procedure. Classic examples include MYCIN (bacterial infection diagnosis) and DENDRAL (chemical analysis).

Architecture of a Rule-Based Expert System

Knowledge Base: a collection of IF-THEN (condition-conclusion) rules encoding expert knowledge, e.g., IF fever AND cough AND body_ache THEN suspect_flu.
Working Memory (Fact Base): the current set of known facts (e.g., observed patient symptoms) which grows as new facts are inferred.
Inference Engine: the reasoning mechanism (forward or backward chaining) that applies rules to the working memory to derive new facts/conclusions.
User Interface: for entering symptoms and displaying the diagnosis with an explanation of how it was reached.

Forward Chaining (Data-Driven Reasoning)

Forward chaining starts from known facts (symptoms) and applies rules whose IF-conditions are satisfied, adding their THEN-conclusions as new facts, repeating until no more rules can fire. It is data-driven and well-suited when many possible conclusions must be derived from a given set of observations -- e.g., 'given these symptoms, what diseases are plausible?'

Backward Chaining (Goal-Driven Reasoning)

Backward chaining starts from a hypothesis/goal (e.g., 'does the patient have Disease X?') and works backward: it finds rules whose THEN-conclusion matches the goal, then recursively tries to prove each of that rule's IF-conditions as sub-goals (asking the user or checking working memory), until all sub-goals are confirmed (goal proved) or a sub-goal cannot be established (goal rejected). It is goal-driven and efficient when testing a specific hypothesis rather than deriving everything that could be concluded.

Example Rule Base (Illustrative)

Rule	IF (Symptoms)	THEN (Conclusion)
R1	fever AND cough AND body_ache	suspect_flu
R2	fever AND rash AND joint_pain	suspect_dengue
R3	cough AND breathlessness AND chest_pain	suspect_pneumonia
R4	sneezing AND runny_nose AND mild_fever	suspect_common_cold
R5	suspect_flu AND high_fever (>102F)	recommend_flu_test

Certainty and Explanation

A practical expert system often attaches a confidence/certainty factor to each rule or conclusion (since symptoms rarely map to a disease with 100% certainty), and provides an explanation facility that shows the chain of rules used to reach a diagnosis, which is important for building user trust in medical applications.

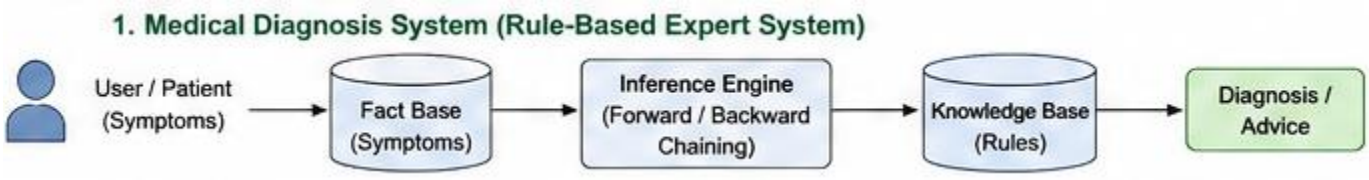
Algorithm Steps -- Forward Chaining

1. Initialize working memory with the entered symptoms.
2. Repeat: scan the rule base for any rule whose IF-conditions are all present in working memory and that has not already fired.
3. If such a rule is found, 'fire' it -- add its THEN-conclusion to working memory as a new fact.
4. Stop when no more rules can fire; report all derived conclusions (candidate diagnoses) as the output.

Algorithm Steps -- Backward Chaining

1. Set the goal to the hypothesis being tested (e.g., 'suspect_flu').
2. Find rule(s) whose THEN matches the goal.
3. For each IF-condition of that rule: if it is already a known fact, treat it as satisfied; otherwise recursively treat it as a new sub-goal and try to prove it (query the user, or search other rules).
4. If all IF-conditions of some rule are satisfied, the goal is PROVED. If no rule can satisfy the goal (and it is not a known fact), the goal is REJECTED.

Diagram



7. Conclusion

A rule-based Medical Diagnosis expert system was implemented with both forward-chaining and backward-chaining inference engines. Given a set of patient symptoms, forward chaining correctly derived all plausible diseases, while backward chaining correctly confirmed or rejected a specific hypothesized disease, demonstrating both data-driven and goal-driven reasoning over the same rule base.

8. Viva Voce Questions

1. What are the main components of a rule-based expert system?
2. Differentiate forward chaining from backward chaining with an example.
3. When would you prefer backward chaining over forward chaining in a diagnostic system?
4. What is a certainty factor, and why is it useful in medical expert systems?
5. What is the explanation facility of an expert system, and why is it important?
6. How does an expert system differ from a simple database lookup?
7. What are the limitations of a purely rule-based (non-probabilistic) diagnosis system?
8. How could Bayesian reasoning improve upon simple rule-based diagnosis?

Assignment C2

Title

Develop a simple rule-based recommendation system (e.g., movie or book suggestion system).

1. Aim

To develop a simple rule-based recommendation system that provides appropriate movie or book suggestions based on user preferences and predefined rules.

2. Prerequisite

Basics of data structures (lists, dictionaries), IF-THEN rule concepts, elementary understanding of similarity/matching.

3. Objective

- To understand the concept and types of recommendation systems used in AI applications.
- To study rule-based (knowledge-based) filtering as a recommendation strategy.
- To design and implement a simple rule-based movie/book recommendation system.
- To analyze the strengths and limitations of rule-based recommendations compared to collaborative/content-based filtering.

4. Outcome

Students will be able to design a rule base linking user preferences (genre, mood, past likes) to catalog items and implement a rule-based recommendation engine that suggests relevant movies/books to a user.

5. Software and Hardware Requirements

Python 3.x / Java / C++ compiler, Windows/Linux OS, 4 GB RAM minimum.

6. Theory

Recommendation Systems -- Overview

Recommendation systems help users discover items (movies, books, products) they are likely to enjoy, out of a large catalog. The three broad approaches are: Content-Based Filtering (recommend items similar to ones the user liked before, based on item attributes), Collaborative Filtering (recommend items liked by similar users, based on the ratings/behaviour of a user community), and Knowledge-Based / Rule-Based Filtering (recommend items using explicit domain rules and user-stated preferences), which is the approach used in this assignment.

Rule-Based (Knowledge-Based) Recommendation

A rule-based recommender encodes explicit IF-THEN rules that map user attributes/preferences to catalog items or item categories. This approach does not require historical ratings data (unlike collaborative filtering) and works well for a 'cold start' -- e.g., a brand-new user with no rating history -- since recommendations are derived directly from stated preferences and domain knowledge.

System Components

User Profile: preferences entered or inferred, e.g., favourite genre, mood, preferred era/language, past liked titles.
Item Catalog: each movie/book tagged with structured attributes -- genre, rating, release year, language, mood-tags.
Rule Base: IF-THEN rules mapping combinations of profile attributes to catalog attributes/items, e.g., IF genre_preference == 'Comedy' AND mood == 'Relaxed' THEN recommend items WHERE genre == 'Comedy' AND rating >= 7.
Matching/Ranking Engine: applies the rules to filter and rank catalog items that satisfy the matched rules.

Example Rule Base (Illustrative)

Rule	IF (User Preference)	THEN (Recommend)
R1	genre == 'Sci-Fi' AND age_group == 'Teen'	PG-13 Sci-Fi titles, rating >= 7
R2	mood == 'Sad' AND genre == 'Drama'	Feel-good Drama titles, rating >= 7.5
R3	liked_book_genre == 'Mystery'	Top-rated Mystery/Thriller titles
R4	language == 'Regional' AND genre == 'Comedy'	Regional-language Comedy titles

Ranking and Fallback

When multiple items satisfy a matched rule, they are ranked by a secondary criterion such as average rating, popularity, or recency, and the top-N items are returned as the recommendation list. If no rule matches the given user profile, the system falls back to a default recommendation (e.g., globally most popular / highest-rated items), so the user always receives a useful response.

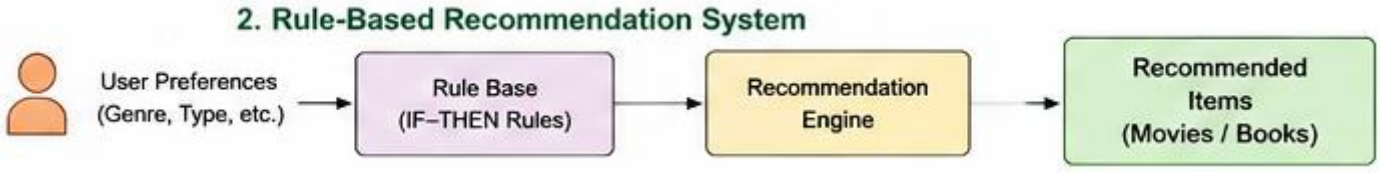
Algorithm Steps

1. Collect the user's stated preferences (genre, mood, past likes, etc.) to build a profile.
2. Match the profile against each rule's IF-condition in the rule base.
3. Collect the set of catalog items satisfying every matched rule's THEN-clause.
4. Rank the collected items by rating/popularity (and remove duplicates).
5. Return the top-N ranked items as the final recommendation list.
6. If no rule matched, return a default popular-items list instead.

Comparison with Other Approaches

Rule-based recommendation is transparent and explainable (the reason for a recommendation is directly traceable to a rule) and needs no historical rating data, but it requires manual rule authoring/maintenance and does not automatically discover subtle patterns the way collaborative filtering or modern machine-learning-based recommenders (e.g., matrix factorization, neural recommenders) can from large-scale user behaviour data.

Diagram



7. Conclusion

A rule-based movie/book recommendation system was implemented using an explicit rule base mapping user preferences to catalog attributes. The system correctly returned relevant, ranked recommendations for various user profiles and used a sensible fallback (popular items) when no rule matched, demonstrating the practicality of knowledge-based recommendation for cold-start users.

8. Viva Voce Questions

1. What are the three main types of recommendation systems?
2. Why is rule-based/knowledge-based filtering well suited to the 'cold start' problem?
3. Differentiate content-based filtering from collaborative filtering.
4. What is the role of the rule base in this recommendation system?
5. How would you rank multiple items that satisfy the same rule?